Reverse-engineering findings · not a bypass

Muse CVM Attestation — Security Review

How the confidential-VM attestation gate is built, what it protects, and whether it can be defeated by modifying the app.

Target	Build	Reviewed	Bundle
Muse.app (“Endo”) v4.1	**1077426479**	2026-09-28	**com.meta.endo**

`binary sha256 029997a6a3ad7f5336dbf96b2ce566e9d126b1b14c4b86c7e3cd87a90a1b6f36 · Developer ID Meta Platforms V9WTTPBFK9 · hardened runtime · NOT FairPlay`

Verdict

The gate that keeps an outside client out is server-side and cryptographic. The attestation check that lives *in the app* protects the client, not the server — turning it off opens no door.

Nothing you can patch in the local binary produces the two things the backend demands: a session `auth_token` minted behind your logged-in session, and a completed Noise handshake against the confidential VM’s real private key. Those are held server-side. So “talk to Muse directly from an external client” stays blocked for the same reason as before — and it is **not** the attestation-verify step that blocks it.

The handshake it rides on

Transport is a Noise_XX_25519_AESGCM_SHA256 session over `wss://<vm>.metaaivm.com/v1/noise`. Three messages, then the channel splits into two AES-GCM ciphers.

msg 1 · client → CVM

`writeMessage1`: client ephemeral X25519 public key (the “client nonce”).

msg 2 · CVM → client

`readMessage2` (≥96 B): server ephemeral + encrypted server static + a **confidential-VM attestation payload**. Pure crypto here — it is decrypted and handed *up*, not judged.

msg 3 · client → CVM

`writeMessage3`: client’s own encrypted static key → `split()` into tx/rx ciphers. Every later frame is AES-GCM under these.

The URL itself carries the server-side credentials: `vm_id` · `auth_token` · `notary_token` · `app_id=hatch-web` · `request_id`.

Two checks, pointed in opposite directions

Client-side · patchable

The attestation-verify (protects *you*)

A trust evaluator confirms the VM you reached is a genuine confidential VM, bound to *this* handshake:

```
if (message2Kind !== "attestation")
    throw "owner_rv_challenge";
await verify({ attestationPayload,
  serverStaticPublicKey, handshakeHash,
  clientNonce });        // else attestation_rejected
```

Strictness is a MobileConfig flag `hatch_web:attestation_enforce_mode` — **0 Off · 1 Staging · 2 Prod** (default 0), with a user override and a downgrade reporter. It’s JS + a JS `verify()`, so it *can* be flipped or removed. Doing so only tells your client “don’t bother checking the VM is real.”

Server-side · not patchable

The access gate (keeps others *out*)

The backend serves the session only when all of these hold — none reproducible from the client:

- `auth_token` minted behind your authenticated session at `POST /api/hatch/token`.
- `notary_token` endorsing the noise session for that `vm_id` (per-CVM notary key; `delegation.` tokens are signed by `getHatchCvmNotarySigningKey(vm_id)`).
- The Noise handshake completing against the CVM's **genuine X25519 static key** — whose shared secret encrypts every frame.

An external party has none of these, and a local patch mints none of them.

What modifying the app would — and would not — achieve

IF YOU…	EFFECT	GETS EXTERNAL ACCESS?
Set `attestation_enforce_mode = 0` / delete the `verify()`	Your client stops confirming the VM is genuine — you lose your own anti-MITM protection.	No
Replay a captured `attestationPayload`	Fails — it's bound to `serverStaticPublicKey` + `handshakeHash` + `clientNonce` of a live handshake.	No
Reuse the app's `app_id` / build a fresh client	Still needs a live `auth_token` + `notary_token` + the CVM's private key.	No
Drive the real logged-in session (browser shim / native paste-and-click)	The genuine client completes the handshake; you ride it. This already works.	Yes (supported)

This is why the standing constraint (don't defeat the attestation, don't patch the signed loader) isn't even the limiting factor for direct access: the limiting factor is **server-held secrets**. Decompilation doesn't yield those.

Threat model

ATTACKER POSITION	AGAINST	OUTCOME
Network MITM between client and CVM	Confidentiality / impersonating the VM	Blocked when enforce ≥ Staging: attestation must verify against the live handshake. If a user flips enforce to Off, this protection is voluntarily dropped.
External client with no session	Reaching the backend RPCs	Blocked: no `auth_token`/`notary_token`, cannot complete the handshake.
Local user modifying their own app	Their own client checks	Can disable the client attestation check; gains no server access from it.
Compromised/fake CVM presenting an `owner_rv_challenge` only	Serving the owner without attesting	Rejected under strict evaluator (`message2Kind !== "attestation"` ⇒ throw).

Evidence

All from the v4.1 web client at `Muse.app/Contents/Resources/hatch/index.html` (minified; symbols quoted verbatim).

SYMBOL	WHAT IT PROVES
`NoiseXXInitiator.readMessage2`	Decrypts server static + tail and returns the attes
`createStrictSharedAgentNoiseTrustEvaluator`	The verify step: requires `message2Kind==="att` `verify({attestationPayload,serverStati` throws `attestation_rejected` / `owner_rv_ch`
`useHatchAttestationEnforcementTier`	Reads `hatch_web:attestation_enforce_moc` resolved tier; downgrades reported to Sentry.
`AttestationEnforcementControlImpl`	User-facing control mapping 0→Off, 1→Staging, 2
`resolveHatchCvmNoiseNotaryToken`	`delegation.` tokens endorsed via `getHatchCvm` CVM notary signing key.
`buildWebSocketUrl`	Handshake URL requires `auth_token`, `notary_t`

Deep dive — what `verify()` actually accepts addendum

The verifier is real, not a stub. It is `SnpAttestationVerifier` — the CVMs are **AMD SEV-SNP** (tcb steppings famId 25 = Milan/Genoa, famId 26 = Turin). `.verify()` → `runAttestationCheck`, which lazy-loads an in-bundle `pipeline` + `policy` module; cert chains are validated with a full X.509/CT library (pkijs). The bundle also carries Intel-TDX parsing code (`tdx`, `mrtd`, `rtmr`), but the instantiated Muse verifier is SNP.

It checks three things, none of which is a pinned image hash:

1 · handshake binding

`SHA-256(binding) == report_data[0:32]` (else `BindingMismatch`); `report_data[32:64]` must be all-zero (`ReportDataTrailNotZero`). Ties the SNP report to this exact handshake.

2 · hardware policy

`allowDebug:false` · `requiredVmpl:0` · per-stepping `tcbFloors` (min bootloader/tee/snp/microcode) · `allowMigration:false` · `denyGuestSmt/denyPlatformSmt` · `requirePageSwapDisable` · `requireAliasCheck`.

3 · transparency + revocation

Image manifest must be a **Cloudflare-endorsed** authenticated manifest (`cloudflareTransparencyPolicy`); else `RevocationPolicyInvalid`. Must be un-revoked (`revocationList[namespace]`) and fresh (`maxAgeMs` / `maxFutureSkewMs`).

The crux: `verify()` does **not** pin a launch-measurement hash in the client. It accepts *any genuine AMD SEV-SNP VM* that meets the hardware floor **and** presents a Cloudflare-endorsed, un-revoked, fresh image manifest bound to the live handshake. Workload identity comes from the **transparency/manifest chain** (rooted in Cloudflare + AMD), not from a hash baked into the app. That is a deliberate design choice: the shipped client never has to be re-released when the CVM image is updated — the endorsement/transparency service is the moving part.

Owner path vs shared-agent path. The owner's own connection runs `activeAttestationVerifier()` → `PRODUCTION_ATTESTATION_VERIFIER` at tier *prod*, `STAGING_…` at *staging*, and `void 0` (no verification) at tier *off* — the shipped default. The shared-agent path uses the separate `strictSharedAgentAttestationVerifier`, which always requires `message2Kind==="attestation"`. Notary token is server-fetched: `POST /api/hatch/noise-notary-token`. So at the default tier the owner's own CVM is *not* attestation-checked client-side; MobileConfig raising the tier to prod turns on full SNP quote verification.

Residual observations

- **Default enforce mode is 0 (Off).** The client's own VM-genuineness check is not on by default in this build; it's raised by MobileConfig (Staging/Prod) and employee tiers. This is a client self-protection posture, not the access gate — but worth noting that anti-MITM verification is opt-in from the shipped default.
- **Employee downgrade path exists** (`ecto1_is_employee` feeds `resolveAttestationTier`, with `reportAttestationDowngrade`) — a rollout/testing affordance, logged when it lowers the tier.
- **Attestation is handshake-bound** (serverStaticPublicKey + handshakeHash + clientNonce), which correctly defeats replay/transplant of a captured attestation.
- **Corrected (see deep dive above):** a client-side measurement policy *is* shipped — an earlier draft of this review had not yet located it. It lives in the code-split `init_policy` module (`productionPolicy` = hardware + transparency + revocation) and is **structural + Cloudflare-transparency-based, not a pinned image hash**. So "what does verify() accept" is now answered: any genuine SEV-SNP VM meeting the hardware floor with a Cloudflare-endorsed, un-revoked, fresh, handshake-bound manifest.

Scope & posture

Defensive reverse-engineering of software on the owner's own machine (DMCA §1201(f) interoperability / security research). This document describes the mechanism and its properties only. No bypass, exploit, patch, or attestation defeat was produced or attempted; the CVM attestation and the signed loader were not modified. The only "talk directly" paths remain the browser-backed shim driving the real logged-in session and native paste-and-click.